COMP90038 Algorithms and Complexity

Graph Traversal

Junhao Gan

Week 4 Lecture 8

Semester 1, 2026

In this lecture, we introduce two arguably the most important fundamental graph traversal algorithms:

- the Depth-First Search (DFS), and
- the Breadth-First Search (BFS).

As can be seen from their names, these two algorithms have different strategy on how to explore the graph.

Stack and Queue

Before we introducing these algorithms, we first revisit two simple data structures which have been briefly introduced in Week 1:

the stack and the queue.

Stack

The **Stack** is a data structure to support **last-in-first-out** item order maintenance. Specifically, it supports the following operations:

- **isEmpty()**: return whether the current stack is empty or not;
- **getTop()**: if the stack is not empty, return the **top** item x, i.e., the *last inserted* item, in the stack; after this operation, x is still in the stack;
- **pop()**: if the stack is not empty, **remove** the top item out from the stack; after this operation, the next last inserted item becomes at the top;
- **push(x)**: insert item x to the stack; after this operation x becomes the top item.

Queue

The **Queue** is a data structure to support **first-in-first-out** item order maintenance. Specifically, it supports the following operations:

- **isEmpty()**: return whether the current queue is empty or not;
- **getTop()**: if the queue is not empty, return the **top** item x, i.e., the *earliest inserted* (existing) item, in the queue; after this operation, x is still in the queue;
- **pop()**: if the queue is not empty, **remove** the top item out from the queue; after this operation, the next earliest inserted item becomes at the top;
- **push(x)**: insert item x to (the end of) the queue;

An Exercise

Let S be an initially empty *stack* and Q an initially empty *queue*.

Consider the following operation sequence:

push(3), push(7), getTop(), pop(), getTop(), push(16), pop(), push(18), push(4), pop(), pop(), pop(), isEmpty()

Exercise:

Perform these operations with the stack S and write down the status of S after every operation.

Do the same thing on the queue Q.

A Unified Graph Traversal Algorithm Framework

Let SQ denote a data structure which is either a *stack* or a *queue*.

Denote our graph traversal algorithm by $\mathcal{A}$.

Given a **connected** graph $G = \langle V, E \rangle$ and a specified source node $s \in V$, $\mathcal{A}(G, s)$ works as follows:

- if the *flags* of nodes have not been initialised,
  - mark every node in V as *unvisited*;
- $SQ \leftarrow \emptyset$; $SQ.push(s)$; mark s as *visited*
- while SQ is *not* empty, do the following:
  - $u \leftarrow SQ.getTop()$;
  - if u has an *unvisited* neighbour v,
    - $SQ.push(v)$; mark v as *visited*;
  - otherwise, **traverse** u and mark u as *traversed*; $SQ.pop()$;

A Unified Graph Traversal Algorithm Framework

Depending on the implementation of SQ, our algorithm $\mathcal{A}$ can be either a *depth-first search* (DFS) or a *breadth-first search* (BFS) on G.

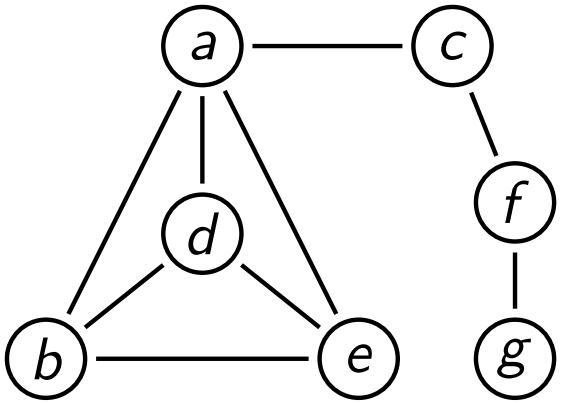
Specifically,

- when SQ is a stack, our algorithm, denoted by $\mathcal{A}_{\text{stack}}$, is a DFS;
- when SQ is a queue, our algorithm, denoted by $\mathcal{A}_{\text{queue}}$, is a BFS.

Depth-First Search: $\mathcal{A}$ with a Stack

$DFS(G, s) = \mathcal{A}_{\text{stack}}(G, s)$:

- if the flags of nodes have not been initialised,
  - mark every node in V as *unvisited*;
- $SQ \leftarrow \emptyset$; $SQ.push(s)$; mark s as *visited*
- while SQ is *not* empty, do the following:
  - $u \leftarrow SQ.getTop()$;
  - if u has an *unvisited* neighbour v,
    - $SQ.push(v)$; mark v as *visited*;
  - otherwise, **traverse** u and mark u as *traversed*; $SQ.pop()$;

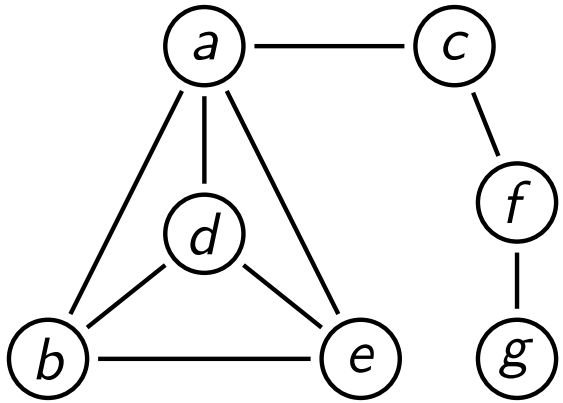


			e						g				
		d	d	d				f	f	f			
	b	b	b	b	b		c	c	c	c	c		
a	a	a	a	a	a	a	a	a	a	a	a	a	
–	–	–	–	–	–	–	–	–	–	–	–	–	–

Depth-First Search: $\mathcal{A}$ with a Stack

Therefore, the node DFS traversal order is:

$$e_{4,1}, d_{3,2}, b_{2,3}, g_{7,4}, f_{6,5}, c_{5,6}, a_{1,7}$$



Following the notations in Levitin's book, the first subscripts give the order in which nodes are pushed, the second the order in which they are popped off the stack.

			e						g				
		d	d	d				f	f	f			
	b	b	b	b	b		c	c	c	c	c		
a	a	a	a	a	a	a	a	a	a	a	a	a	
–	–	–	–	–	–	–	–	–	–	–	–	–	–

Depth-First Search on Disconnected Graphs

What if G is not connected?

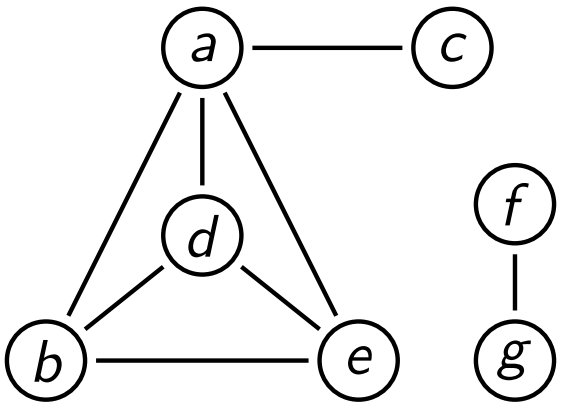
When the graph G is not connected, we can run $\mathcal{A}_{\text{stack}}$ on an arbitrary **un-traversed** node, until every node in G is traversed.

Specifically, the **general DFS algorithm**, $DFS(G)$, works as follows:

- while G has a node *un-traversed*, do the following:
  - pick an arbitrary un-traversed node u (with zero in-degree if G is directed);
  - run $\mathcal{A}_{\text{stack}}(G, u)$;

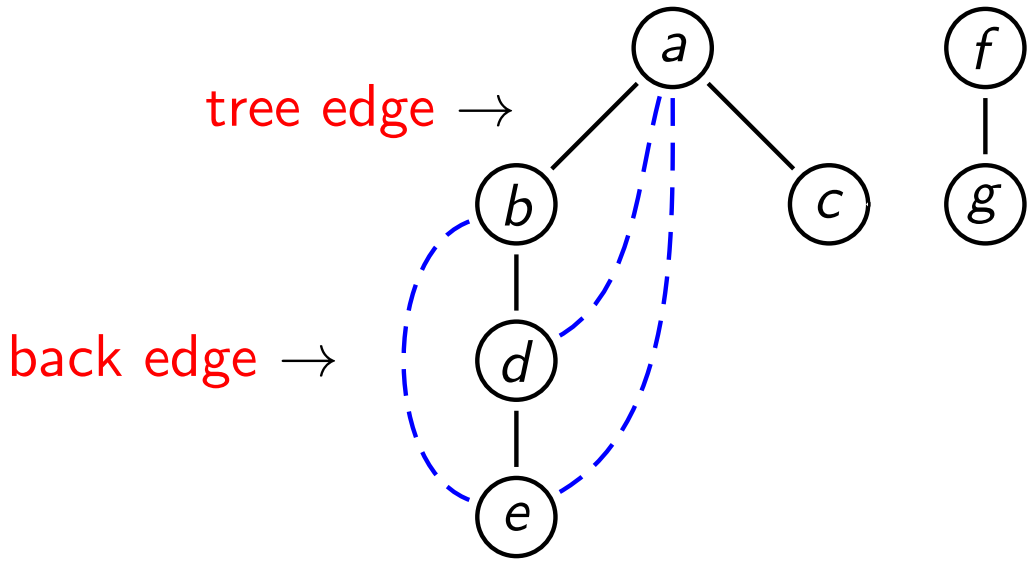
The general $DFS(G)$ works on both undirected and directed graphs.

Depth-First Search: The DFS Forest



Another useful tool for depicting a DFS traversal is the DFS tree (for a connected graph), where an edge (u, v) is a *tree edge* in the DFS tree **if and only if** when v is pushed into SQ, u is at the top of the stack. Otherwise, (u, v) is a *back edge*.

More generally, we get a DFS forest:



More Implementation Details

To maintain the labels of the nodes whether they are *visited* (rsp., *traversed*) or not:

- we can use an integer array *flag* of length $|V|$, the i^{th} of which corresponds to the i^{th} node in G;
- each integer has only *three* possible values: 0 means *unvisited*, 1 means *visited* and 2 means *traversed*.

The array *flag* has the following usages:

- The label of each node whether it is visited or not can be easily checked and marked.
- By scanning *flag*, we can easily find the **next** un-traversed node in G and start $\mathcal{A}_{\text{stack}}$ from there.
- Further, when *flag* is completely scanned, we know that all the nodes have been traversed.

More Implementation Details

To ensure the adjacency list (or the row in the adjacency matrix) of each node v, denoted by $adj(v)$, is scanned *only once*, we can maintain a *pointer* to record where the *last scanned* node is in $adj(v)$.

Therefore, the next *unvisited* neighbour of v can be found by keep scanning $adj(v)$ from the current pointer.

If $adj(v)$ is completely scanned, then we know that there is no *un-visited* neighbour of v.

Depth-First Search: Analysis

First, the initialization of *flag* takes $O(|V|)$ time, and finding all un-traversed nodes requires scanning *flag* only once, and hence, its cost is also bounded by $O(|V|)$.

Second, as we discussed in Week 1, each operation of Stack can be performed in $O(1)$ time. Observe that except for the $O(1)$ number of Stack operations outside the *while-loop*, each Stack operation in *while-loop* actually comes with some $adj(v)$ scanning.

By the fact that each $adj(v)$ is scanned only once, the overall cost of changing the node labels and the Stack operations is bounded by $O(\sum_{v \in V} |adj(v)|)$.

Depth-First Search: Analysis

Putting the above together, the overall cost of the general $DFS(G)$ is bounded by $O(|V| + \sum_{v \in V} |adj(v)|)$.

- If G is represented by an **adjacency matrix**, then each $|adj(v)| = |V|$. Therefore, the overall running time cost is bounded by $O(|V|^2)$.

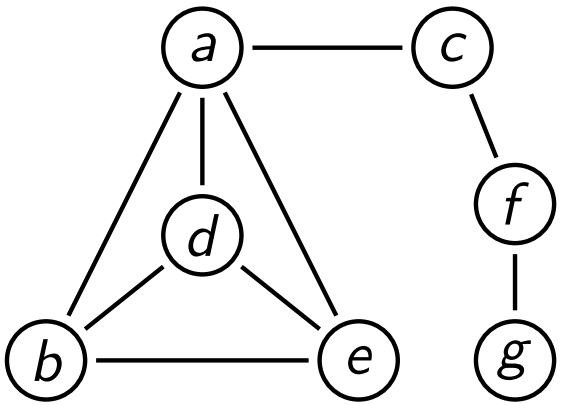
- If G is represented by **adjacency lists**,
  - $|adj(v)| = deg(v)$ if G is undirected, where $deg(v)$ is the degree of v;
  - $|adj(v)| = deg_{\text{out}}(v)$ if G is directed, where $deg_{\text{out}}(v)$ is the out-degree of v;
  - it can verified that, in either case, $\sum_{v \in V} |adj(v)| = O(|E|)$.

  Thus, the overall running time cost is bounded by $O(|V| + |E|)$.

Breadth-First Search: $\mathcal{A}$ with a Queue

$BFS(G, s) = \mathcal{A}_{\text{queue}}(G, s)$:

- if the flags of nodes have not been initialised,
  - mark every node in V as *unvisited*;
- $SQ \leftarrow \emptyset$; $SQ.push(s)$; mark s as *visited*
- while SQ is *not* empty, do the following:
  - $u \leftarrow SQ.getTop()$;
  - if u has an *unvisited* neighbour v,
    - $SQ.push(v)$; mark v as *visited*;
  - otherwise, **traverse** u and mark u as *traversed*; $SQ.pop()$;



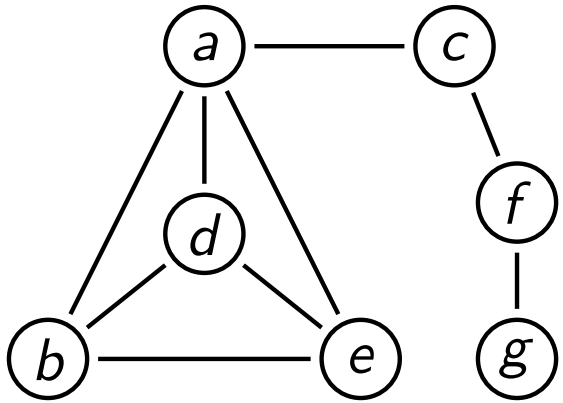
				e									
			d	d	e		f						
		c	c	c	d	e	e	f					
	b	b	b	b	c	d	d	e	f		g		
a	a	a	a	a	b	c	c	d	e	f	f	g	
—	—	—	—	—	—	—	—	—	—	—	—	—	—

Breadth-First Search: $\mathcal{A}$ with a Queue

Therefore, the node BFS traversal order is:

$$a_{1,1}, b_{2,2}, c_{3,3}, d_{4,4}, e_{5,5}, f_{6,6}, g_{7,7}$$

The first subscripts give the order in which nodes are pushed, the second the order in which they are popped off the queue.



				e									
			d	d	e		f						
		c	c	c	d	e	e	f					
	b	b	b	b	c	d	d	e	f		g		
a	a	a	a	a	b	c	c	d	e	f	f	g	
–	–	–	–	–	–	–	–	–	–	–	–	–	–

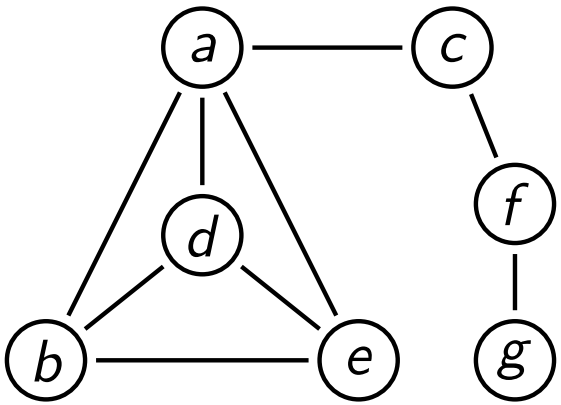
Breadth-First Search on Disconnected Graphs

The **general BFS algorithm**, $BFS(G)$, works as follows:

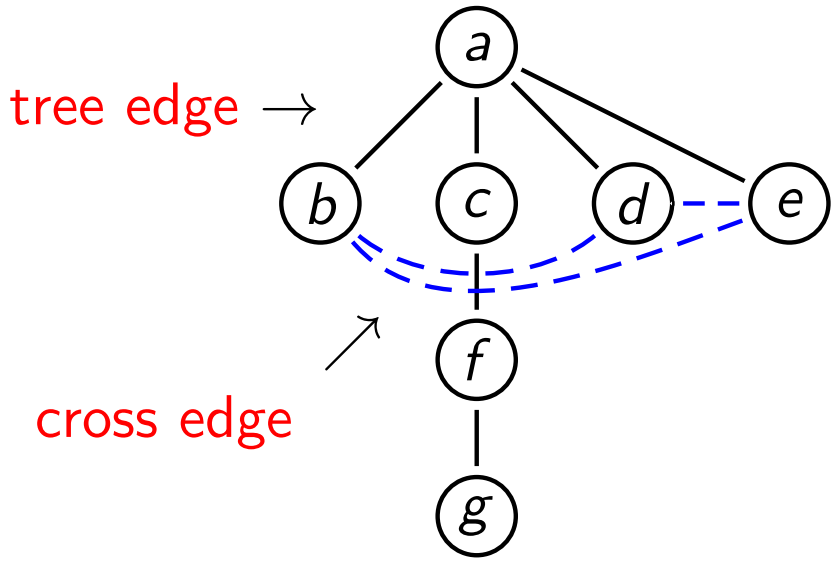
- while G has a node *un-traversed*, do the following:
  - pick an arbitrary un-traversed node u (with zero in-degree if G is directed);
  - run $\mathcal{A}_{\text{queue}}(G, u)$;

The general $BFS(G)$ works on both undirected and directed graphs.

The Breadth-First Search Forest



Here is the BFS tree for the example:



In general, we may get a BFS forest.

Moreover, in an *unweighted* graph G, the BFS tree rooted at s is also known as a **single-source shortest-path tree** from s in G.

Breadth-First Search: Analysis

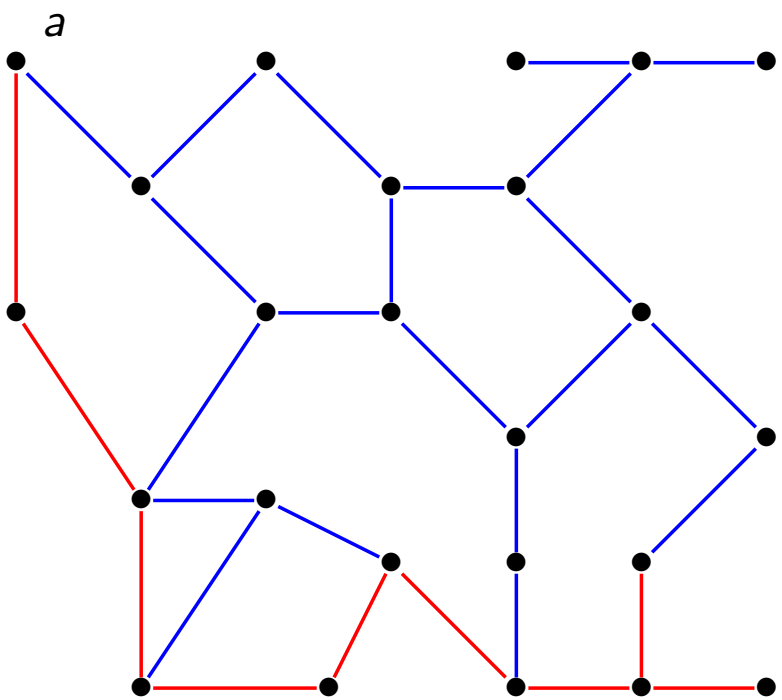
Since the BFS only differs from the DFS in a queue versus a stack, as we discussed in Week 1, each operation of a queue can be performed in $O(1)$, the overall running time cost bound of the BFS is the same as that of the DFS.

That is,

- if G is represented by an adjacency matrix, then the overall cost is bounded by $O(|V|^2)$;
- if G is represented by adjacency lists, then the overall cost is bounded by $O(|V| + |E|)$.

DFS vs BFS

Typical depth-first search:



Typical breadth-first search: